

Report On

Design and Implementation of a Blockchain Based Secure Identity Authentication System

Submitted to the Department of Information Technology



Submitted By

Pallab Mandal (2022ITB001)

Harshit Kumar (2022ITB011)

Subhranil Nandy (2022ITB012)

B.Tech, Information Technology

Indian Institute of Engineering Science and Technology, Shibpur

Under the guidance of

Dr. Indrajit Banerjee

Professor, Department of Information Technology,
Indian Institute of Engineering Science and Technology , Shibpur

May 2026

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude to Dr. Indrajit Banerjee, our esteemed supervisor, for providing us the invaluable opportunity to undertake this Final Year Project on “Design and Implementation of a Blockchain Based Secure Identity Authentication System”. Dr. Indrajit Banerjee’s expertise, encouragement, and constructive feedback have been instrumental in shaping the conceptualization, methodology, and execution of our project.

We also extend our heartfelt appreciation to the Department of Information Technology, Indian Institute of Engineering Science and Technology, Shibpur, for providing the foundational academic training and research environment that enabled us to contribute effectively to this interdisciplinary work.

This report reflects the progress of our focused work and the valuable guidance received from Dr. Indrajit Banerjee and the academic community at our institution. Their expertise and support in the domain of blockchain technology and secure identity systems continue to shape the scope and direction of this ongoing project on Design and Implementation of a Blockchain Based Secure Identity Authentication System.

Pallab Mandal
2022ITB001

Harshit Kumar
2022ITB011

Subhranil Nandy
2022ITB012

Department Of Information Technology
Indian Institute Of Engineering Science And Technology
Shibpur, Howrah 711103

CERTIFICATE

This is to certify that the project entitled “Design and Implementation of a Blockchain Based Secure Identity Authentication System” submitted by Pallab Mandal (2022ITB001), Harshit Kumar (2022ITB011) and Subhranil Nandy (2022ITB012) to Indian Institute of Engineering Science and Technology, Shibpur, towards partial fulfillment of the requirement for the award of degree of Bachelor of Technology in Information Technology is a record of bonafide work carried out by them under my supervision. This dissertation, in my opinion, is worthy of consideration for the purpose for which it has been submitted, and it fulfills the requirements of the regulations of this institute. The results incorporated in this dissertation are original and have not been submitted to any other university or institute for the award of any degree or diploma.

Dr. Indrajit Banerjee
Professor
Department of Information Technology
Indian Institute of Engineering Science and Technology
Shibpur, Howrah, India 711103
May, 2026

Dr. Tuhina Samanta
Head of the department
Department of Information Technology
Indian Institute of Engineering Science and Technology
Shibpur, Howrah, India 711103
May, 2026

INDEX

ACKNOWLEDGEMENT	2
CERTIFICATE	5
ABSTRACT	6
CHAPTER 1: INTRODUCTION & PROBLEM DEFINITION	7
1.1 Introduction	7
1.2 Problem Statement	7
1.3 Objectives	8
1.4 Project Scope	9
1.4.1 In Scope	9
1.4.2 Out of Scope	9
CHAPTER 2: LITERATURE SURVEY & FEASIBILITY ANALYSIS	10
2.1 Literature Review	10
2.1.1 Paper 1	10
2.1.2 Paper 2	11
2.2 Proposed System	12
2.2.1 Improvements Over Existing Solutions	12
2.3 Feasibility Study	13
2.3.1 Technical Feasibility	13
2.3.2 Economic Feasibility	14
2.3.3 Operational Feasibility	14
CHAPTER 3: SYSTEM REQUIREMENT ANALYSIS	15
3.1 Vulnerabilities in Traditional Verification	15
3.2 Cryptographic Hashing & Off-Chain Storage	16
3.3 Blockchain & Smart Contracts	17

CHAPTER 4: METHODOLOGY AND EVALUATION	20
4.1 System Architecture	20
4.2 Smart Contract Architecture	20
4.3 Trust and Security Model	21
4.4 Operational Workflow	21
4.4.1 DID Registration	22
4.4.2 Credential Issuance	22
4.4.3 Verification and Audit	22
4.5 Evaluation Methods	23
 CHAPTER 5: RESULTS AND DISCUSSION	 25
 CHAPTER 6: CHALLENGES AND LEARNINGS	 30
6.1 Challenges Faced	30
6.2 Learnings and Takeaways	31
 CHAPTER 7: FUTURE PLANS AND ENHANCEMENTS	 32
7.1 Decentralized Storage (IPFS)	33
7.2 Zero-Knowledge Proofs (ZKPs)	33
7.3 Mainnet Deployment & Layer-2 Scaling	33
7.4 Institutional Interoperability	34
 CHAPTER 8: CONCLUSION	 35
 CHAPTER 9: REFERENCES	 36

ABSTRACT

The rapid growth of digital services has increased the demand for secure, transparent, and tamper proof identity authentication systems. Traditional authentication mechanisms often suffer from issues such as centralized control, data breaches, identity theft, and lack of transparency. To address these challenges, this project titled “Design and Implementation of a Blockchain Based Secure Identity Authentication System” presents a decentralized framework for secure academic identity and certificate verification using blockchain technology.

The proposed system utilizes Ethereum smart contracts to create a secure and immutable platform for identity registration, credential issuance, and verification. The system integrates blockchain features such as decentralization, cryptographic hashing, immutability, and role based access control to ensure data integrity and prevent unauthorized modifications. MetaMask wallet authentication is used to authorize blockchain transactions, while Ganache is used as the local blockchain development environment. The frontend is developed using React.js and the backend is implemented using Node.js and Express.js.

The system enables authorized issuers to issue academic certificates securely by storing the cryptographic hash of certificate data on the blockchain, while the complete certificate details remain stored off chain. QR code based credential sharing and verification mechanisms are also implemented to simplify validation processes for institutions and employers. Additionally, an immutable audit trail is maintained for all certificate operations, ensuring transparency and traceability.

The developed solution demonstrates how blockchain technology can enhance the security, authenticity, and reliability of digital identity authentication systems while reducing the risks associated with traditional centralized verification methods. The project provides a scalable and cost effective approach for secure credential management and verification in academic environments.

Chapter 1: Introduction & Problem Definition

1.1 Introduction

In the current educational ecosystem, the processes for issuing and verifying academic credentials remain heavily reliant on legacy methods. Educational institutions issue thousands of certificates each year, yet most of these documents exist either as physical paper or as standard digital PDFs. These traditional formats are vulnerable to unauthorized alteration, loss, and duplication, making it difficult for external organizations to confirm their authenticity.

To address these trust and verification challenges, modern software systems are incorporating blockchain technology. A blockchain acts as a distributed, tamper resistant ledger that operates without a central authority. By deploying smart contracts on a blockchain, developers can create Decentralized Identifiers (DIDs) that securely link a specific user or institution to a verifiable digital identity.

This project, titled "**Design and Implementation of a Blockchain Based Secure Identity Authentication System,**" focuses on building an Academic Certificate Verification Portal. By leveraging Ethereum based smart contracts rather than a centralized database, the system ensures that academic records are securely verifiable and highly resistant to unauthorized tampering.

1.2 Problem Statement

The current infrastructure used by universities to manage and verify academic records faces several practical engineering and operational challenges:

- **High Verification Latency:** When an employer or background screening agency needs to verify a graduate's certificate, they must contact the issuing institution via email, phone, or a manual portal. This manual verification process is slow, administrative, and can take weeks to complete.
- **Simple Document Forgery:** Digital certificates issued as standard PDFs can be easily altered using basic image editing tools. Unscrupulous actors can change names, degrees, or grades without leaving an obvious digital trace on the file itself.
- **Central Database Vulnerabilities:** Storing student records in a traditional centralized database creates a single point of failure. If an unauthorized attacker

gains access to the database, or if an internal user abuses administrative privileges, academic records can be altered directly in the tables.

- **Server Side Private Key Risks:** Many web applications perform digital signing on a central backend server using a shared institutional key. If the server is compromised by malware or a data breach, the private key can be stolen, allowing attackers to generate fraudulent certificates that appear authentic.

1.3 Objectives

The primary goal of this project is to develop a decentralized web application that enables decentralized issuance and validation of academic certificates. The specific engineering objectives include:

- **Implement an Identity Mapping Layer:** Develop a DIDRegistry smart contract that maps cryptographic wallet addresses to authorized college profiles and student identities.
- **Enforce Wallet Based Transaction Signing:** Design an architecture where the backend server remains read only, forcing all data altering transactions to be approved and signed directly by users via the MetaMask browser extension.
- **Ensure Data Integrity via Hashing:** Utilize a hybrid off chain/on chain storage model that calculates a keccak256 hash of the certificate data, making unauthorized modifications easily detectable through hash verification.
- **Apply On Chain Access Control:** Restrict the certificate issuance feature exclusively to verified institution accounts by using role based access modifiers within the smart contracts.
- **Maintain a System Audit Trail:** Log critical system events permanently within an independent AuditLog smart contract to establish a transparent history of operations.
- **Provide User Friendly Verification Routes:** Integrate a dynamic QR code generation tool in the frontend interface so that third parties can scan and instantly view the validation status of a credential.

1.4 Project Scope

To complete the project within the academic timeline and maintain focus on the core system architecture, the functional boundaries have been explicitly defined as follows.

1.4.1 In Scope (What the System Does)

- **Smart Contract Development:** Designing, compiling, and testing three core Solidity contracts: DIDRegistry, CredentialRegistry, and AuditLog.
- **Local Testnet Integration:** Configuring and deploying the smart contracts to a local Ganache network using Hardhat for development and testing (Chain ID: 1337).
- **Standardized Certificate Fields:** Implementation of a specific data model for academic certificates containing: Student Wallet Address, Student Name, College Name, Course, Grade, and Passing Year.
- **Read Only Backend Architecture:** An Express.js backend optimized to read data from the blockchain while intentionally returning an HTTP 410 Gone status for write routes to protect against server side signing.
- **Frontend Integrity Verification:** A client side verification script that re hashes user provided certificate data to check it against the blockchain, clearly failing if a single character or grade has been modified.

1.4.2 Out of Scope (Explicit System Constraints)

Note for Evaluators: The following features are excluded from this project by design to manage resource consumption, minimize system complexity, and keep the implementation tightly scoped to a local academic prototype.

- **No Decentralized File Storage (e.g., IPFS):** The system does not implement a decentralized file storage layer. Instead, it relies on passing the structured certificate payload as a standard off chain data object, recording only the validation hash on chain.
- **No Advanced Cryptographic Proofs (e.g., ZKPs):** Data validation is achieved using direct keccak256 hash comparisons. Complex cryptographic frameworks like zero knowledge proofs are not implemented.
- **No Live Public Network Deployment:** The portal is developed and evaluated strictly within a local blockchain testing environment (Ganache). Migration to a live public testnet or mainnet is omitted due to fluctuating gas cost management.
- **No Custodial Wallet Management:** The application does not manage, generate, or store private keys or seed phrases for the user. Private key security and wallet connections are handled entirely by the external MetaMask browser extension.

CHAPTER 2: LITERATURE SURVEY & FEASIBILITY ANALYSIS

2.1 Literature Review

To build a reliable digital credential system, it is essential to analyze existing research methodologies, frameworks, and implementations in the educational blockchain domain. This section reviews two core academic publications and one open source framework, detailing their specific strengths and technical shortcomings.

Paper 1: Blockchain based Decentralised Micro Accreditation for Lifelong Learning

Authors: Alexander Mikroyannidis, Aisling Third, and John Domingue

This research paper proposes a decentralized micro accreditation framework for lifelong learning using blockchain technology. The system focuses on issuing and validating digital learning achievements through blockchain based credentials. The authors utilize decentralized technologies to allow learners to own and share their educational achievements securely without depending entirely on centralized institutions.

The proposed framework improves transparency and portability of educational credentials while reducing the risk of certificate forgery. Blockchain immutability ensures that issued credentials remain tamper resistant and verifiable throughout a learner's lifetime.

Strengths

- Supports decentralized ownership of educational credentials
- Provides tamper resistant certificate verification
- Encourages lifelong learning portability
- Reduces dependency on centralized educational authorities

Limitations

- Primarily focused on micro accreditation rather than full academic identity systems
- Limited discussion on practical implementation challenges
- Does not provide detailed audit trail mechanisms

- User authentication and issuer authorization mechanisms are not deeply addressed

Paper 2: Blockchain Assisted Self Sovereign Identities on Education: A Survey

Authors: Weilin Chan, Keke Gai, Jing Yu, and Liehuang Zhu

This survey paper discusses the application of Self Sovereign Identity (SSI) frameworks in educational systems using blockchain technology. The paper analyzes various blockchain based identity management approaches and highlights how decentralized identities can improve privacy, security, and control over educational records.

The authors examine different SSI architectures, decentralized identifiers (DIDs), cryptographic verification techniques, and blockchain integration models for educational environments. The paper also discusses challenges such as scalability, interoperability, privacy protection, and adoption barriers.

Strengths

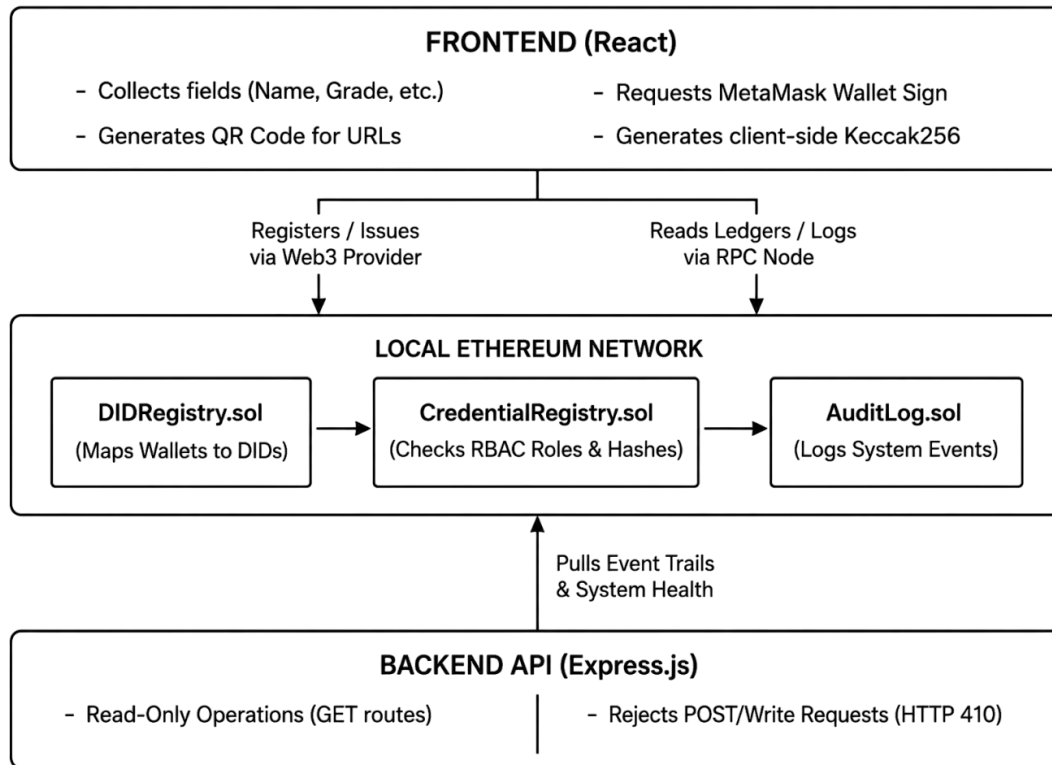
- Provides a comprehensive overview of blockchain based educational identity systems
- Explains the importance of decentralized identity management
- Highlights privacy and ownership benefits of SSI frameworks
- Discusses current challenges and future opportunities in blockchain based education systems

Limitations

- Primarily theoretical and survey oriented
- Does not provide a complete implementation model
- Limited focus on practical certificate verification workflows
- Does not address QR based verification or immutable audit logging in detail

2.2 Proposed System

The system developed in this project directly addresses the operational gaps and safety vulnerabilities identified in the literature survey. Instead of relying on complex RDF graphs or insecure server side signing mechanisms, our portal balances performance and data security through a clean, decoupled design.



Improvements Over Existing Solutions:

- 1. Wallet Enforced Signing (Mitigating Server Breaches):** Unlike traditional frameworks (such as Blockcerts) that sign transactions using server side keys, our system utilizes a read only backend. Any data altering route on the Express server returns an HTTP 410 Gone error. All certificate issuances, verifications, and registration actions must be signed on the client side using the user's specific MetaMask wallet, protecting the infrastructure from single point server breaches.
- 2. Hybrid Off Chain/On Chain Performance (Mitigating Gas Costs):** Rather than committing large structured datasets to the ledger—which would incur unsustainable gas costs—our system implements a hybrid commitment model. The Express backend handles fast, low overhead read requests for specific document details stored off chain, while the CredentialRegistry contract stores only the unique keccak256 payload hash (proofHash).
- 3. Simplified User Facing Verification (Mitigating Operational Complexity):** Instead of forcing verifiers to run complex client side parsing tools (as seen in MerQL frameworks), the frontend auto generates dynamic QR codes containing the unique credential ID and verification URL. A third party verifier simply scans

the code to trigger an automated on chain hash comparison, making the application accessible to non technical users.

4. **On Chain Access Governance:** Built in Role Based Access Control (RBAC) via contract variables ensures that only explicitly approved institutional wallets (granted permission by the contract owner using `addIssuer`) can publish academic records, preventing unauthorized credential production.

2.3 Feasibility Study

An engineering analysis was performed to verify that the project can be effectively built, deployed, and sustained within real world constraints.

2.3.1 Technical Feasibility

The software components and development tools required for this project are highly mature, widely adopted, and fully compatible with modern hardware environments:

- **Smart Contract Layer:** Solidity serves as a robust language for writing custom access logic. Hardhat provides an integrated development environment for compilation, execution testing, and automated scripts, while Ganache offers a local Ethereum runtime environment (Chain ID: 1337) that accurately mimics a live network.
- **Application Layer:** The combination of React.js for the user interface and Express.js for the backend routing represents a proven JavaScript stack. MetaMask provides an industry standard browser API (`window.ethereum`) that integrates smoothly with the application using the `Ethers.js` library.
- **Hardware Requirements:** Development, testing, and execution do not require heavy specialized infrastructure; standard consumer workstations with Node.js installed are capable of running the local blockchain node, backend server, and frontend client concurrently.

2.3.2 Economic Feasibility

The project is economically viable for both development and long term production use:

- **Development Phase:** The entire development lifecycle relies exclusively on open source frameworks, software development kits, and local testing tools. No financial budget is required to purchase software licenses, server hosting, or active network gas fees.

- **Production Operations:** Because the system employs a hybrid storage model where full descriptive parameters stay off chain and only thin cryptographic hashes are sent to the smart contracts, transaction execution costs are kept to a minimum. If migrated to public production environments, alternative Layer 2 networks can keep operation costs to fractions of a cent per certificate.

2.3.3 Operational Feasibility

The application is designed to align with existing user habits, ensuring low resistance during adoption:

- **For Institutional Administrators:** The system features a clear UI form containing standard fields (Student Name, College, Course, Grade, Year). Issuing a certificate matches standard web workflows, with the added step of confirming the transaction pop up inside the MetaMask extension.
- **For External Verifiers (Employers):** Verifiers do not need to download specialized tools, configure blockchain nodes, or understand smart contract interactions. Scanning the generated QR code directs them to a clean verification window that clearly displays a green success state or a red tampering alert, hiding the underlying cryptographic logic behind a standard user experience.

CHAPTER 3: PROBLEM DEFINITION & SYSTEM REQUIREMENT ANALYSIS

3.1 Vulnerabilities in Traditional Academic Certificate Verification

A critical problem in modern academic credentialing is ensuring the authenticity of certificates and preventing fraudulent activities such as grade tampering, forged certificates, or unauthorized data modification. In traditional academic certificate verification systems, certificate records are usually stored in centralized databases controlled by a single institution or administrative body. Although such systems are easy to manage, they are vulnerable to unauthorized changes in critical fields such as student name, college name, course name, grade, passing year, and certificate status.

Centralized systems also introduce the risk of credential inflation, where unauthorized users may issue invalid certificates under an institution's name if access control is weak. In addition, traditional database based systems often act as a single point of failure. An internal user with database privileges or an external attacker who gains access to the server may alter records directly without producing a transparent and immutable audit trail.

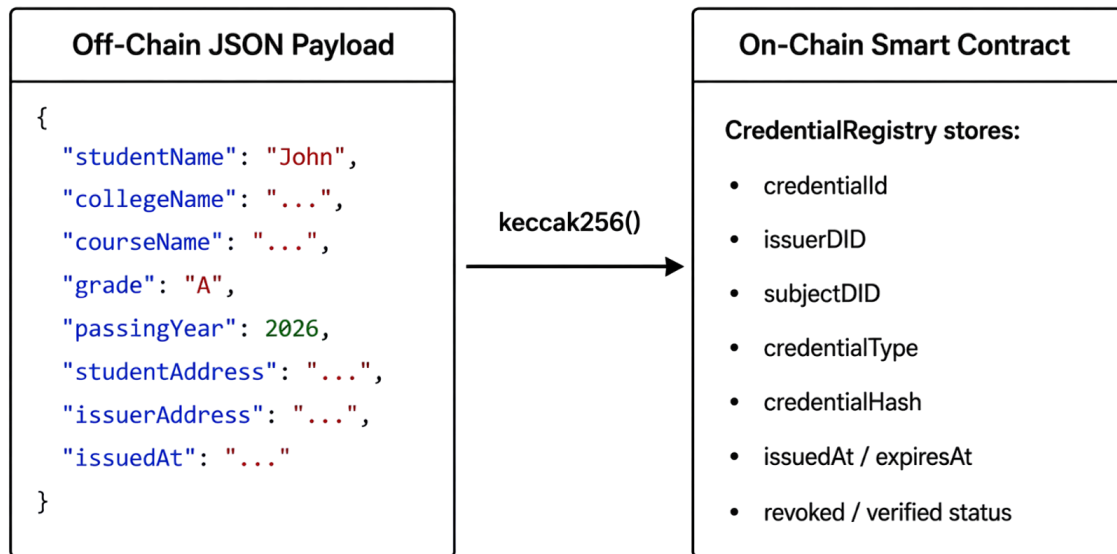
These limitations create a strong need for a verification architecture that can ensure data integrity, restrict certificate issuance to authorized institutional accounts, and maintain a permanent record of certificate related operations. Therefore, the proposed system uses blockchain, smart contracts, cryptographic hashing, decentralized identity concepts, and wallet based transaction signing to improve trust, transparency, and tamper detection in academic certificate verification.

3.2 Importance of Cryptographic Hashing and Off Chain Storage Techniques

Storing complete academic certificate data directly on a blockchain is inefficient because blockchain storage is expensive and not suitable for large or frequently accessed textual data. To reduce storage cost while preserving integrity, the proposed system stores the complete certificate payload off chain and records only its cryptographic hash on chain.

In this system, the academic certificate data is first structured as a JSON payload containing fields such as student name, college name, course name, grade, passing year,

student wallet address, issuer address, and issue timestamp. A keccak256 hash is then generated from this JSON payload. This 32 byte hash is stored in the smart contract as credentialHash.



The hash acts as a cryptographic commitment to the original certificate data. If any field in the off chain certificate payload is changed after issuance, even by a single character, the newly generated hash will be different from the hash stored on chain. During verification, the frontend recomputes the hash of the available certificate payload and compares it with the credentialHash stored in the CredentialRegistry smart contract.

If both hashes match, the certificate data is considered unchanged. If the hashes do not match, the system detects possible tampering and rejects the certificate verification. This mechanism helps detect unauthorized changes to important academic data such as grades, student names, courses, colleges, or passing years.

In the implemented prototype, the off chain certificate payload is stored in the browser's local storage for later integrity checks. In a production environment, this off chain data could be stored in a secure institutional database, IPFS, or another trusted decentralized storage system, while the blockchain would continue to store only the hash commitment.

3.3 Importance of Progressing to Blockchain and Smart Contracts

Progressing from a centralized database model to a blockchain based model changes the trust structure of academic certificate verification. Instead of relying only on an institution's internal database, the proposed system uses Ethereum compatible smart contracts deployed on a local Ganache blockchain network. This provides a transparent and tamper resistant foundation for issuing and verifying academic credentials.

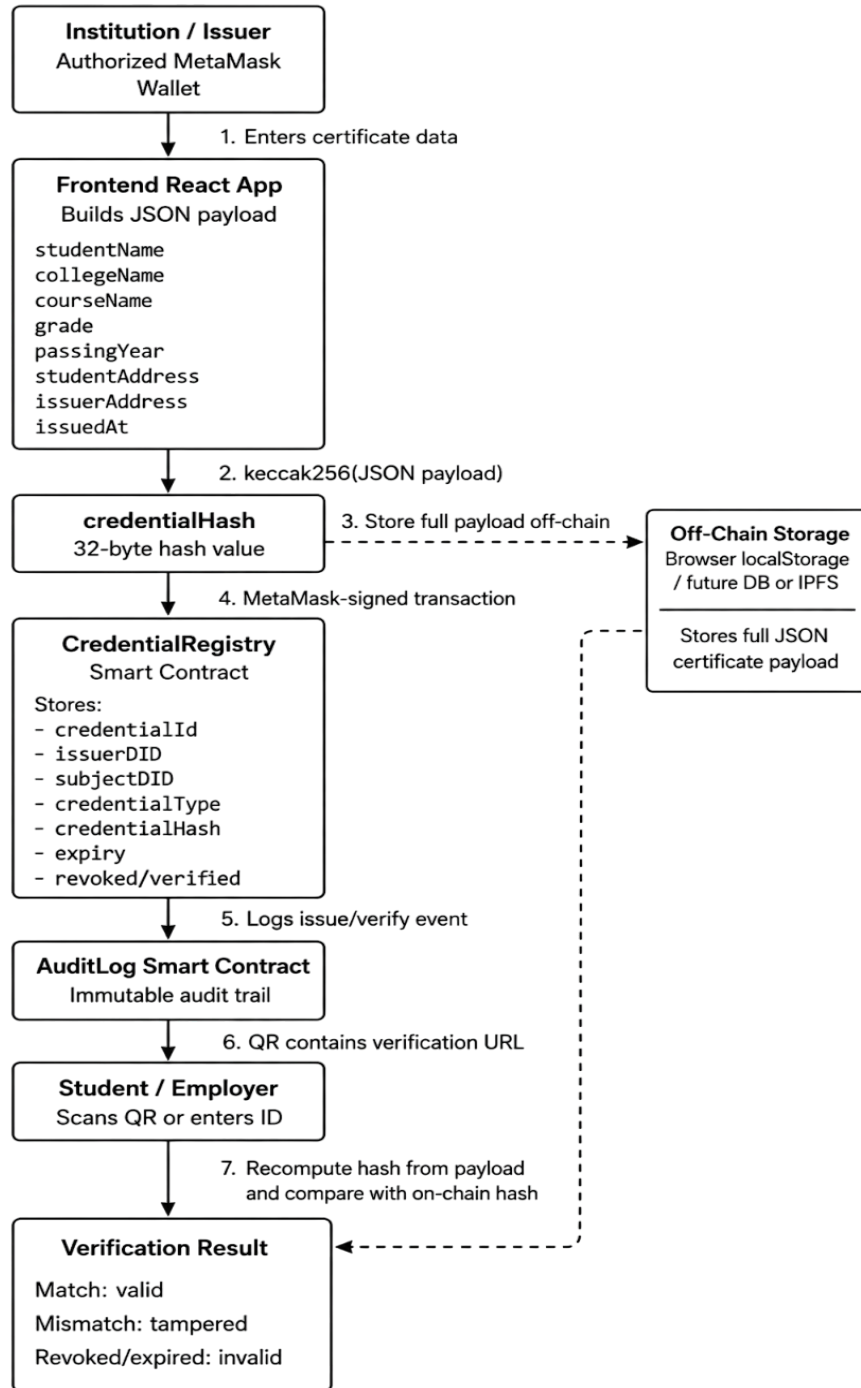
The system uses three main smart contracts: DIDRegistry, CredentialRegistry, and AuditLog. The DIDRegistry contract manages decentralized identity records and maps wallet addresses to DID based identity information. The CredentialRegistry contract manages certificate issuance, verification, revocation, credential hashes, expiry status, and issuer authorization. The AuditLog contract records important events such as DID creation, credential issuance, credential verification, and credential revocation.

A major advantage of this architecture is immutable auditability. Once a certificate is issued or verified, the related event is recorded on chain and cannot be silently modified like a traditional database row. This provides a transparent trail of important certificate operations.

The system also introduces on chain role based access control. Only authorized issuer accounts can issue academic certificates. The contract owner can authorize institutional accounts using the addIssuer function, and the CredentialRegistry contract uses an onlyIssuer modifier to ensure that unauthorized users cannot create valid credentials. This prevents unauthorized certificate generation and reduces the risk of credential inflation.

Another important improvement is the use of client side wallet based signing. The backend does not sign blockchain write transactions. Instead, write operations such as DID registration, certificate issuance, and certificate verification are performed through the frontend using MetaMask. The backend write routes are intentionally disabled and return HTTP 410 Gone, ensuring that sensitive blockchain transactions are not signed by a centralized backend server. This reduces server side signing risks and gives users direct control over blockchain transaction approval.

The system also improves the certificate sharing and verification process through QR codes. After a certificate is issued, the frontend generates a QR code containing a verification URL with the unique credentialId. A student can share this QR code with an employer or third party verifier. The verifier can scan the QR code, retrieve the credential ID, and initiate the verification process through the frontend.



Thus, the proposed system provides a more secure and transparent method for academic certificate verification. By combining cryptographic hashing, off chain storage, smart contracts, decentralized identity, issuer based access control, MetaMask transaction signing, immutable audit logs, and QR based verification, the system addresses major weaknesses of traditional certificate verification systems.

CHAPTER 4: METHODOLOGY AND EVALUATION

The proposed methodology for the Academic Certificate Verification Portal is implemented through a decentralized architecture. The system shifts away from traditional centralized databases, utilizing Ethereum smart contracts to ensure the immutability and tamper evident nature of academic credentials. The methodology is executed through distinct architectural layers and operational phases: System Architecture setup, Smart Contract deployment, Trust & Security formulation, and a Three Phase Operational Workflow (Registration, Issuance, and Verification).

4.1 System Architecture Overview

The system operates on a multi tier architecture to securely bridge the end user with the blockchain network. The high level components include:

Frontend Interface: A React based web application (utilizing ethers.js) that serves as the primary user interface. It handles form collections, off chain payload generation, and crucially, interacts with the MetaMask browser extension to facilitate client side transaction signing.

Backend API: An Express.js backend API that acts strictly as a read only aggregation layer. It provides endpoints to query Decentralized Identifiers (DIDs), fetch credential details, and retrieve system wide or DID specific audit trails.

Blockchain Node: A local Ganache Ethereum node (configured to Chain ID 1337 and RPC <http://127.0.0.1:7545>) that processes transactions, mines blocks, and hosts the smart contracts.

4.2 Smart Contract Architecture

The core logic and state of the system are governed by three interacting Solidity smart contracts deployed on the local blockchain:

DIDRegistry: Responsible for decentralized identity management. It registers credential schema hashes and DID documents. It fundamentally maps a user's Ethereum wallet

address to their unique DID string, ensuring that all subsequent credentials are tied to a cryptographically verifiable identity.

AuditLog:

Acts as an immutable on chain ledger for critical system actions. It persists records of system interactions—such as DID_CREATED, CREDENTIAL_ISSUED, and CREDENTIAL_VERIFIED—along with the actor's wallet address and a timestamp.

CredentialRegistry:

The central hub for academic credential operations. Deployed with the AuditLog address for automatic event logging, this contract handles the issuance, verification, and revocation of credentials. It enforces Role Based Access Control (RBAC) through an isIssuer mapping, ensuring that only the authorized owner (e.g., the university administration) can add issuers and issue certificates via the onlyIssuer modifier.

4.3 Trust and Security Model

To eliminate vulnerabilities associated with centralized key management, the system employs a strict client side trust model and a hash based integrity mechanism.

Wallet Based Write Signing: The backend API does not construct wallets from user provided private keys, nor does it sign write transactions. All write routes (e.g., /api/v1/credential/issue) are intentionally disabled (returning HTTP 410). Instead, the user's wallet directly signs write transactions in the browser via MetaMask, ensuring non repudiation.

Hash Based Credential Integrity: To optimize gas costs and preserve data privacy, the system does not store full certificate data on chain. When a credential is created, the frontend builds a JSON payload (containing the student name, college, course, grade, and passing year) and computes its Keccak 256 hash. Only this hash (proofHash or credentialHash) is stored on the blockchain. Any tampering with the off chain data (e.g., altering a grade) will yield a mismatched hash during verification, thereby alerting the verifier.

4.4 Operational Workflow

The system's execution is divided into three primary runtime phases, representing the lifecycle of an academic credential:

4.4.1 Phase 1: Identity (DID) Registration

In the initial phase, a student or entity binds their Ethereum wallet to a decentralized identity. The frontend automatically registers a basic schema (such as name and email) if it is missing. The user then submits their DID data, and the React app calls the `DIDRegistry.registerDID` function directly using the MetaMask signer. Once the transaction is mined on Ganache, the DID hash (`keccak256(didString)`) is finalized, and a `DID_CREATED` event is logged to the AuditLog.

4.4.2 Phase 2: Credential Issuance

Authorized administrators issue academic certificates in this phase.

The issuer inputs the student's wallet address alongside academic details (name, college, course, grade, passing year). The frontend constructs an off chain JSON payload and computes its `keccak256` hash. The frontend resolves both the issuer and student DIDs and invokes the `CredentialRegistry.issueCredential` contract function via MetaMask.

The smart contract verifies the issuer's RBAC privileges, creates a credential record (with `status verified=false`), logs a `CREDENTIAL_ISSUED` action, and emits an event containing the `credentialId`. Upon successful issuance, the frontend generates a shareable QR Code containing a verification URL with the specific `credentialId`. The original payload is securely stored in the issuing browser's `localStorage` for future integrity checks.

4.4.3 Phase 3: Credential Verification and Audit

This phase allows third parties (e.g., employers) to independently verify a student's academic claims.

The verifier scans the QR code or manually inputs the `credentialId` into the portal. The system fetches the on chain credential record, retrieving the stored `credentialHash`. If the original payload is available to the browser, the frontend recomputes the hash of the payload and compares it against the on chain hash utilizing the `read only verifyCredentialIntegrity` helper. If the hashes match (confirming no data tampering has

occurred), a transaction is signed to call `verifyCredential`, setting the on chain status to `verified=true`. Simultaneously, the system logs a `CREDENTIAL_VERIFIED` action to the `AuditLog`. The verifier can also utilize the backend read APIs to inspect the full immutable audit trail of the credential or the specific DID.

4.5 Method of Evaluation

To ensure the reliability, security, and accuracy of the Academic Certificate Verification Portal, the system is evaluated through a combination of automated smart contract testing, end to end integration testing, and cryptographic integrity validation.

4.5.1 Smart Contract Unit Testing

The core logic of the blockchain layer is evaluated using the Hardhat development environment. Automated test scripts are written to validate state changes and access controls. The tests evaluate:

RBAC Enforcement: Verifying that only authorized addresses (issuers) can successfully invoke the `issueCredential` function, preventing unauthorized credential generation.

Event Emission: Ensuring that `CredentialIssued` and logging events are accurately fired and recorded upon successful transactions, verifying the audit trail's reliability.

Credential State: Validating that credentials correctly initialize with a `verified=false` state and correctly transition to `verified=true` upon successful integrity checks.

4.5.2 End to End System Testing

The overall system architecture is evaluated via end to end (E2E) testing utilizing a local Ganache network and the MetaMask browser extension. This tests the complete user journey:

Frontend Wallet Integration: Evaluating the ability of the React frontend to successfully prompt MetaMask for transaction signing without relying on backend private key management.

API Interactions: Validating that the backend Express API successfully aggregates read only data, such as querying decentralized identifiers and retrieving comprehensive audit trails.

QR Code Workflow: Testing the generation and resolution of QR codes to ensure they correctly map to the respective on chain credential records and load the verification dashboard correctly.

4.5.3 Cryptographic Integrity Validation

The most critical evaluation metric of the system is its ability to detect and prevent data tampering. This is tested through:

Hash Matching: Comparing the keccak256 hash of the localized off chain JSON payload against the immutable credentialHash stored on the smart contract.

Tamper Detection: Systematically altering certificate data (e.g., modifying a student's grade or passing year in the payload) to ensure the system consistently rejects the verification attempt and flags it as a "data mismatch," thereby successfully preventing fraudulent academic claims.

CHAPTER 5: RESULT

The proposed blockchain based academic certificate verification system was successfully implemented and tested in a local development environment using Ganache, Ethereum smart contracts, MetaMask, a React frontend, and an Express backend. The system achieved the main objective of issuing academic certificates as blockchain backed credentials and verifying them using tamper proof hash validation.

The user interface was divided into three main phases: DID registration, academic certificate issuance and verification, and audit trail inspection. In the first phase, a student identity was registered as a Decentralized Identifier (DID) and linked with a wallet address. In the second phase, an authorized issuer issued an academic certificate containing student name, college, course, grade, and passing year. The complete certificate data was stored off chain, while its keccak256 hash was stored on chain to ensure integrity. A QR code was also generated for each issued certificate, allowing the certificate to be shared and verified easily.

The screenshot displays the 'Academic Certificate Verification Portal' interface. At the top, a navigation bar shows 'SYSTEM / CERTIFICATE VERIFICATION'. Below this, the main header area includes the title 'Academic Certificate Verification Portal' and a table with account details:

ACCOUNT:	BALANCE:	NETWORK ID:
0x4bbb965e...fdb0	100 ETH	1337

Below the header, there are three tabs: 'PHASE 1: REGISTRATION & ISSUANCE' (active), 'PHASE 2: AUTHENTICATION & VERIFICATION', and 'PHASE 3: ACCESS & AUDIT'. A success message is displayed: 'Success: DID registered! Hash: 0x361132a4a54c1fca5937c986e4607f69e0f9399fa54ee59c37a12c21efc51549'. The main content area for Phase 1 is titled 'PHASE 1: REGISTRATION & ISSUANCE' and contains the instruction 'Register your Decentralized Identifier (DID) and create verifiable credentials'. A form field labeled 'DID STRING (E.G., DID:BLOCKCHAIN:123ABC)' contains the text 'DID:2022ITB001'. Below the form is a 'REGISTER DID' button.

Fig: Phase 1: DID registration

SYSTEM / CERTIFICATE VERIFICATION

Academic Certificate Verification Portal

ACCOUNT:
0x18a33bc7...600f
BALANCE:
99.99 ETH
NETWORK ID:
1337

PHASE 1: REGISTRATION & ISSUANCE

PHASE 2: AUTHENTICATION & VERIFICATION

PHASE 3: ACCESS & AUDIT

Success: DID registered! Hash: 0x4bbacc0f52dc429c6fc0d71390fc1c524f3ebbb2a5bb7121c44b72b811fc6871

PHASE 2: AUTHENTICATION & VERIFICATION

Issue, verify, and audit academic certificates using on-chain credential hashes

STUDENT WALLET ADDRESS

STUDENT NAME

COLLEGE NAME

COURSE

GRADE

PASSING YEAR

ISSUE ACADEMIC CERTIFICATE

Fig: Phase 2: Certificate issuance by the Institute

ISSUED CERTIFICATE

CREDENTIAL ID: 0x14955840fbcd25e883fdf2ce2729b0f430eb14e6dbe9debc5c8259526915ca9a



```

{
  "studentName": "Harshit",
  "collegeName": "IIEST",
  "courseName": "IT",
  "grade": "9",
  "passingYear": "2026",
  "studentAddress": "0x48bb965EDCF61140Ac1D8Dfe610F5c1fcD85fdb0",
  "issuerAddress": "0x18a33bc79d5b1e45b08856b7e96438150198600f",
  "issuedAt": "2026-05-19T12:22:03.710Z"
}

```

Fig: The issued certificate

During verification, the system accepted either a credential ID or a QR based verification URL. The frontend recomputed the hash of the certificate payload and compared it with the hash stored on the blockchain. If both hashes matched, the certificate was successfully verified. This confirmed that any modification in certificate details, such as

grade, student name, course, or passing year, would result in a hash mismatch and the system would detect possible tampering.

The screenshot displays a web interface for verifying a credential. At the top, there is a label "CREDENTIAL ID OR QR URL" above a text input field containing a long hexadecimal string: "0x14955840fbcd25e883fdf2ce2729b0f430eb14e6dbe9debc5c8259526915ca9a". Below the input field is a dark button labeled "VERIFY & CHECK INTEGRITY". Underneath the button, the section "VERIFIED CREDENTIAL JSON" contains a large text area displaying a JSON object. The JSON object includes fields for credential ID, issuer DID, subject DID, credential type, hash, issued and expiration dates, revocation status, verification status, and a payload with student and course details.

```
{
  "credentialId": "0x14955840fbcd25e883fdf2ce2729b0f430eb14e6dbe9debc5c8259526915ca9a",
  "issuerDID": "0x4bbacc0f52dc429c6fc0d71390fc1c524f3ebbb2a5bb7121c44b72b811fc6871",
  "subjectDID": "0x361132a4a54c1fca5937c986e4607f69e0f9399fa54ee59c37a12c21efc51549",
  "credentialType": "ACADEMIC_CERTIFICATE",
  "credentialHash": "0xc12a7258383c345489bca19b9ef89233bbb5c69b74da6aec5ece955d4d83e3e2",
  "issuedAt": "1779193334",
  "expiresAt": "2094553334",
  "isRevoked": false,
  "isVerified": true,
  "integrityVerified": true,
  "payload": {
    "studentName": "Harshit",
    "collegeName": "IIEST",
    "courseName": "IT",
    "grade": "9",
    "passingYear": "2025",
    "studentAddress": "0x488bb965EDCF61140Ac1D8Dfe610F5c1fcD85fdb0",
    "issuerAddress": "0x18A33bc79d5b1e45b08856b7e96438150198600f",
    "issuedAt": "2026-05-19T12:22:03.710Z"
  }
}
```

Fig: The verified credential json

The audit trail module also worked successfully. Every important credential operation, such as certificate issuance and verification, was recorded on chain through the audit contract. These records were retrieved through backend APIs and displayed in the frontend. This provided transparency and traceability for certificate related actions.

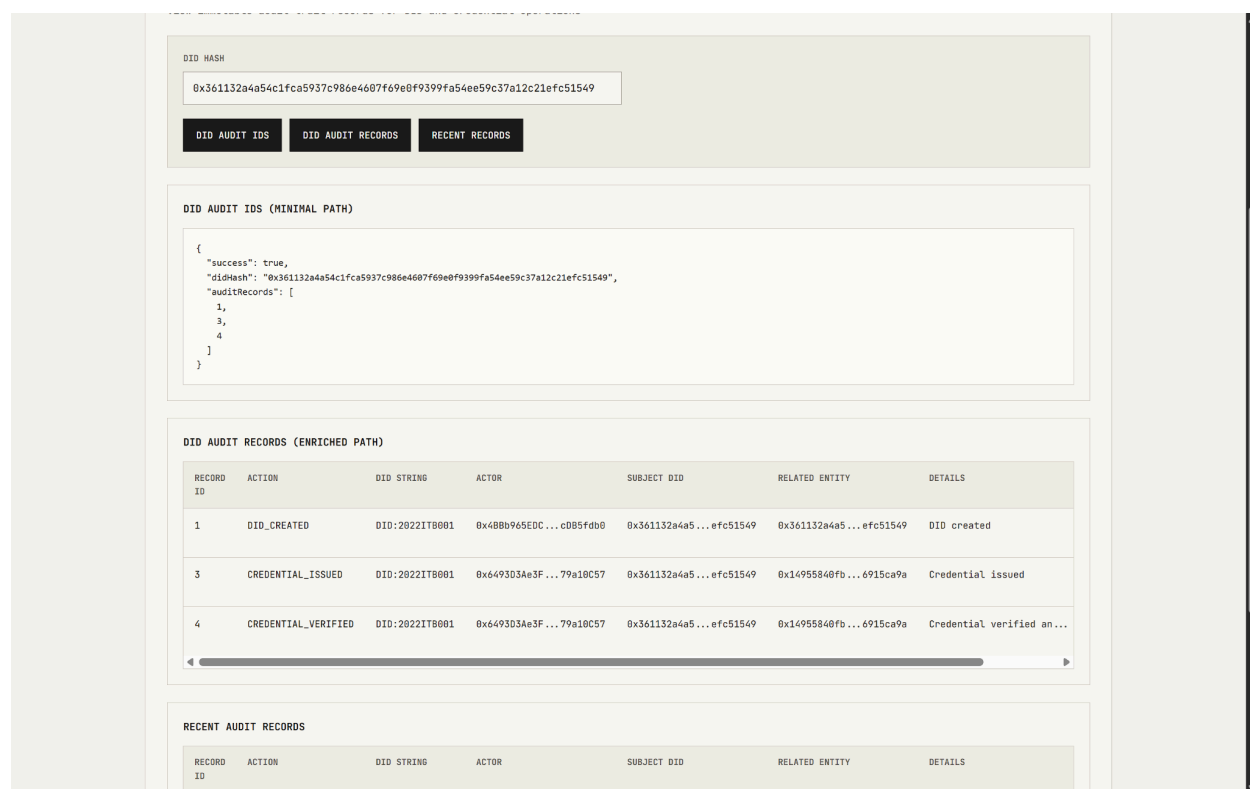


Fig: Audit log

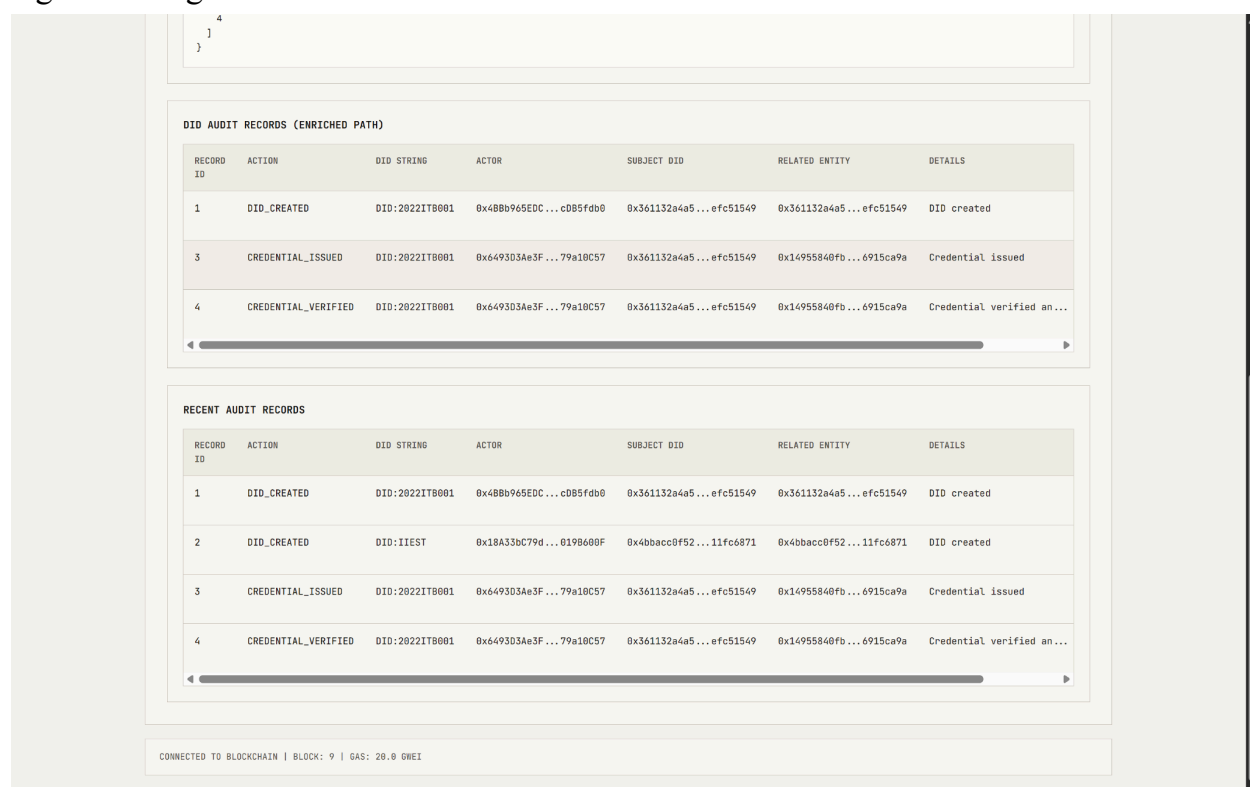


Fig: Audit records

Performance results were collected over 10 runs for each major operation. The DID registration transaction took an average of 268.84 ms, certificate issuance took 420.02 ms, and credential verification took 380.38 ms. The average gas usage was 216,891 gas for DID registration, 432,400 gas for certificate issuance, and 304,101 gas for credential verification. These values show that certificate issuance is the most expensive operation because it stores credential data references and records audit information on chain.

Operation	Average Latency	Average Gas Used
DID Registration	268.84 ms	216,891
Credential Issue	420.02 ms	432,400
Credential Verification	380.38 ms	304,101

Backend read APIs also performed correctly with no errors during testing. Basic read operations such as health check, DID lookup, and credential lookup completed in under 50 ms on average. Audit related read operations took slightly longer, especially recent audit retrieval, which averaged 384.10 ms, because it involves fetching and aggregating multiple audit records.

Backend Endpoint	Average Latency
GET /health	48.05 ms
GET /did/:didHash	48.74 ms
GET /credential/:credentialId	44.81 ms
GET /audit/did/:didHash/ids	35.54 ms
GET /audit/did/:didHash	151.48 ms
GET /audit/credential/:credentialId	147.86 ms
GET /audit/recent/:limit	384.10 ms

The smart contract test case also passed successfully, confirming that the credential issuance functionality worked as expected. Overall, the results demonstrate that the system can register identities, issue blockchain backed academic certificates, verify certificate integrity, generate QR based verification links, and maintain an immutable audit trail. The implementation is suitable for a local proof of concept and shows how blockchain can improve trust, transparency, and tamper resistance in academic certificate verification.

CHAPTER 6: CHALLENGES AND LEARNINGS

6.1 Challenges Faced in System Development

The development of a decentralized, blockchain based application introduces several paradigm shifts compared to traditional web development. Throughout the implementation of the Academic Certificate Verification Portal, several architectural and technical challenges were encountered and addressed:

Decentralized Key Management and Trust Model:

One of the most significant initial challenges was designing a secure trust model. Early iterations often tempt developers to manage wallet private keys on the backend server for smoother user experiences. However, this violates the core tenets of decentralization and introduces a massive security vulnerability. The challenge was successfully migrating all write operations to the client side, strictly enforcing MetaMask wallet based signing in the React frontend, and intentionally disabling backend write APIs (returning HTTP 410) to prevent centralized key management.

On Chain Data Storage Constraints and Gas Optimization:

Storing extensive text data—such as full student names, university details, and course descriptions—directly on the Ethereum blockchain is highly inefficient and incurs prohibitive gas costs. Furthermore, storing raw academic records on a public ledger raises privacy concerns. Overcoming this required implementing a hybrid storage architecture where only a Keccak 256 hash of the certificate payload (proofHash) is stored immutably on chain, while the actual JSON data resides off chain.

Payload Availability for Integrity Verification:

A direct consequence of the hybrid storage model is the dependency on off chain data availability during the verification phase. Currently, the system stores the original JSON payload in the issuing browser's localStorage. The challenge arises during third party verification: if the verifier does not have access to the original payload, the frontend cannot recompute the hash to check against the smart contract. In such cases, the system must fallback to simply verifying the existence and validity of the on chain hash without performing a full payload integrity check.

Zero Knowledge Proof (ZKP) Implementation Complexity:

While the initial scope considered utilizing true Zero Knowledge Proofs for credential verification to maximize privacy, the mathematical complexity and computational overhead of implementing zk SNARKs or zk STARKs on a local Ganache testnet proved challenging within the project constraints. The system was thus adapted to use robust, cryptographic hash based integrity checks as a practical and secure alternative.

Web3 User Experience (UX) and State Management:

Bridging the gap between a traditional web interface and a Web3 environment posed UX challenges. Handling asynchronous MetaMask wallet connection prompts, ensuring the user is on the correct network (Ganache Chain ID 1337), parsing raw blockchain events to extract credential IDs, and generating shareable QR codes required meticulous state management in the React frontend.

6.2 Learnings and Key Takeaways

The lifecycle of this project provided profound insights into both blockchain architecture and full stack integration:

Importance of Role Based Access Control (RBAC) in Smart Contracts:

We learned that deploying a smart contract is only half the battle; securing its execution environment is paramount. Implementing the `isIssuer` mapping and `onlyIssuer` modifier in the `CredentialRegistry` contract highlighted the necessity of strict on chain access controls to prevent fraudulent credential inflation by unauthorized wallet addresses.

Separation of Read and Write Concerns in dApps:

The project solidified the understanding of how Decentralized Applications (dApps) should be structured. By utilizing the Express.js backend strictly as a read only aggregation and caching layer (for audit trails and DID queries) and reserving the frontend for all state changing blockchain writes, the system achieved a highly secure and scalable architecture.

Immutability as a Double Edged Sword:

Working with Solidity contracts reinforced the reality of blockchain immutability. Unlike centralized databases where errors can be easily patched via a SQL update, flawed smart contract logic is permanent once deployed. This emphasized the critical need for rigorous local testing (using Hardhat test scripts) prior to deployment.

Practical Cryptography Application:

The process of generating off chain JSON payloads, stringifying them, and applying keccak256 hashing provided hands on experience with cryptographic commitments. We learned how a single character change in a student's grade cascades into a completely different hash, effectively utilizing the blockchain as an incorruptible digital notary.

Future Scalability Considerations:

The reliance on browser localStorage for off chain data highlighted a critical learning regarding decentralized storage. For the system to scale to a production environment, we recognized the necessity of integrating distributed file systems like IPFS (InterPlanetary File System) or decentralized databases to ensure payload availability across all verifying parties without relying on local machine states.

CHAPTER 7: FUTURE PLANS

While the current implementation provides a secure, tamper evident framework on a local blockchain, strategic enhancements are necessary for production scale. Future phases will focus on complete decentralization, advanced cryptographic privacy, and real world deployment.

7.1 Transitioning to Decentralized Storage (IPFS)

Currently, the system relies on the issuer's local browser storage for off chain JSON certificate payloads. We plan to integrate the InterPlanetary File System (IPFS) for decentralized, highly available data hosting.

Mechanism: Certificate payloads will be encrypted and uploaded to IPFS. The resulting Content Identifier (CID) will be stored on the Ethereum smart contract alongside the credential hash.

Benefit: Verifiers can fetch payloads directly from IPFS via the CID, enabling automated, universally accessible integrity checks without relying on the issuer's localized data.

7.2 Implementation of Zero Knowledge Proofs (ZKPs)

The current hash based verification requires exposing the entire certificate payload to prove its integrity. To enhance data privacy and comply with regulations like GDPR, we will incorporate zk SNARKs.

Mechanism & Benefit: Zero Knowledge Proofs enable a student to mathematically prove a specific claim (e.g., graduating from a specific program or holding a $GPA > 3.0$) without revealing the underlying sensitive data. This facilitates selective disclosure and advances Self Sovereign Identity (SSI) capabilities.

7.3 Mainnet Deployment and Layer 2 Scaling

The application is currently optimized for local testing (Ganache). Moving to production requires migrating to public networks.

Migration: Smart contracts will be deployed to a public testnet (e.g., Sepolia) for rigorous security auditing before a final Ethereum Mainnet deployment.

Layer 2 (L2) Integration: To mitigate high Ethereum gas fees, the system will integrate L2 scaling solutions like Polygon or Arbitrum, ensuring rapid, cost effective transactions while inheriting Ethereum's base layer security.

7.4 Institutional Interoperability

For widespread adoption, the portal must support interoperability across educational institutions globally.

Standardization: We will adopt W3C Verifiable Credentials (VCs) standards to ensure certificates are universally readable and compliant across third party systems.

Consortium Dashboard: A multi tenant administrative dashboard will be developed, allowing a consortium of universities to manage their own issuers and share a unified DIDRegistry.

CHAPTER 8: CONCLUSION

In conclusion, the development of the Blockchain Based Academic Certificate Verification Portal successfully addresses the critical vulnerabilities inherent in traditional, centralized credential management systems. By leveraging Ethereum smart contracts, this project demonstrates a robust, decentralized architecture that guarantees the immutability, transparency, and tamper evident nature of academic records.

Throughout the implementation, we successfully transitioned from traditional backend centric trust models to a fully decentralized paradigm. By enforcing client side transaction signing via MetaMask and removing private key management from the backend, the system established a strictly secure, non repudiable workflow. Furthermore, the integration of Role Based Access Control (RBAC) ensures that only authorized institutional entities can issue certificates, effectively preventing credential inflation and unauthorized modifications.

The utilization of a hybrid storage model—storing the Keccak 256 hash (proofHash) of the certificate on chain while keeping the payload off chain—proved to be an optimal balance between preserving data privacy and minimizing blockchain gas costs. The seamless generation and scanning of QR codes bridge the gap between complex Web3 infrastructure and end user accessibility, allowing employers to verify academic claims in seconds with cryptographically backed certainty.

Looking forward, the roadmap for this project, including the integration of the InterPlanetary File System (IPFS) and Zero Knowledge Proofs (zk SNARKs), will further elevate the system into a globally scalable, privacy preserving standard for Self Sovereign Identity (SSI). Ultimately, this project signifies a meaningful step toward eliminating academic fraud, empowering students with ownership of their credentials, and restoring absolute trust in educational qualifications.

CHAPTER 9: REFERENCES

1. Chan, Weilin, Keke Gai, Jing Yu, and Liehuang Zhu. 2025. "Blockchain-Assisted Self-Sovereign Identities on Education: A Survey" *Blockchains* 3, no. 1: 3.
<https://doi.org/10.3390/blockchains3010003>
2. Mikroyannidis, Alexander, Aisling Third, and John Domingue. 2025. "Blockchain-Based Decentralised Micro-Accreditation for Lifelong Learning." *Interactive Learning Environments* 33 (3): 2201–15.
doi:10.1080/10494820.2024.2401485.
3. Srivastava, Sandeep, Deepshikha Agarwal, and Brijesh Chaurasia. "Secure decentralized identity management using blockchain." In 2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp. 1355-1360. IEEE, 2023.
4. Hammi, Mohamed Tahar, Patrick Bellot, and Ahmed Serhrouchni. "BCTrust: A decentralized authentication blockchain-based mechanism." In 2018 IEEE wireless communications and networking conference (WCNC), pp. 1-6. IEEE, 2018.
5. Liu, Yuan, Zheng Zhao, Guibing Guo, Xingwei Wang, Zhenhua Tan, and Shuang Wang. "An identity management system based on blockchain." In 2017 15th Annual Conference on Privacy, Security and Trust (PST), pp. 44-4409. IEEE, 2017.
6. Bouafia, Khawla, and Mahammad Gulalov. "Blockchain Solutions for authorization and authentication." *Procedia Computer Science* 237 (2024): 115-122.
7. Sporny, Manu, Dave Longley, and David Chadwick. "Verifiable credentials data model v1. 1. W3C Recommendation." World Wide Web Consortium (W3C) (2022).